

NAIJASHOP DBT DATA ENGINEERING WORKFLOW

Student Teaching Guide: From Raw Data to Business Insights

PostgreSQL / Neon Cloud • dbt • SQL • Data Engineering

Practical project based on the NaijaShop database

What Is dbt?

dbt stands for **data build tool**. It is a tool that helps data teams transform data inside a database using SQL.

dbt does not replace SQL.

Instead, it gives us a **structured way to organize, manage, test, document, and run SQL transformations.**

The key idea for students is: SQL is the language; dbt is the framework that helps us turn **many** SQL transformations into a maintainable data engineering project.

Can I Just Use SQL?

Yes. You can connect directly to Neon PostgreSQL and write SQL such as:

```
SELECT
  customer_id,
  COUNT(*) AS total_orders
FROM orders
GROUP BY customer_id;
```

PostgreSQL can execute this query without dbt. The reason we use dbt is not because SQL cannot do the work. The reason is that a real project can contain hundreds of SQL transformations, and we need a reliable way to organize dependencies, test data quality, reuse transformations, document models, and rebuild the pipeline.

SQL	dbt
Language used to query and transform data	Framework for managing SQL transformations
Can run queries directly	Builds reusable models
Does not automatically manage dependencies	Understands dependencies through ref()
Testing must be managed separately	Provides dbt testing workflows
Can become difficult to manage at scale	Provides structure, lineage, documentation and repeatability

Installing dbt

- In your terminal on VScode, *pip install dbt-postgres*
- Then, *dbt init naijashop_dbt*
- Move into that folder, *cd naijashop_dbt*
- Configure your connection (Your database credentials, e.g, host, username, password, etc)
- Test your connection, *dbt debug*

Our NaijaShop Project

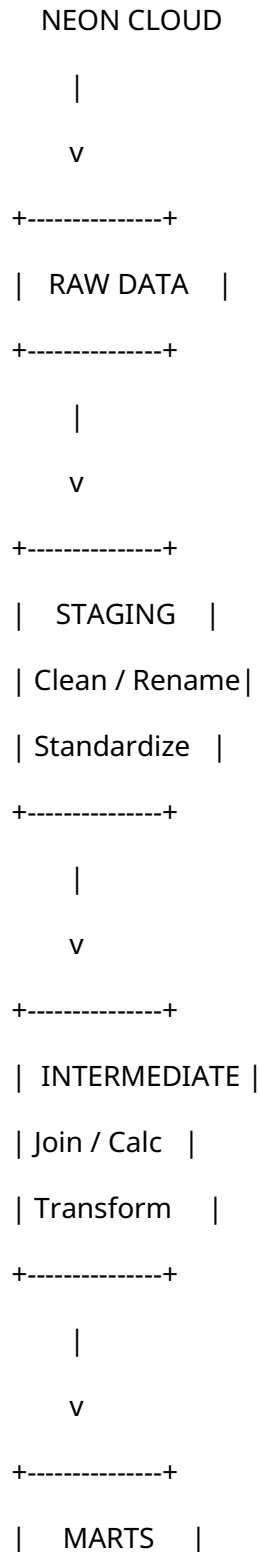
The NaijaShop project is connected to Neon Cloud PostgreSQL. The database contains customer, order, product, payment, exchange-rate and marketing data. Our goal is to transform these raw tables into clean, tested, business-ready datasets that can later feed Power BI or another BI tool.

Current Source Tables

Table	Important columns
customers	customer_id, full_name, email, city, state, signup_date
exchange_rate	_airbyte_raw_id, _airbyte_extracted_at, _airbyte_meta, _airbyte_generation_id, to, from, rate, amount, result, success, change24h, updatedAt, changePct24h
my_first_dbt_model	id
my_second_dbt_model	id
naijashop_marketing_ad_spend_csv	_airbyte_raw_id, _airbyte_extracted_at, _airbyte_meta, _airbyte_generation_id, clicks, channel, amount_ngn, spend_date, campaign_id, conversions, campaign_name
order_items	order_item_id, order_id, product_id, quantity, unit_price
orders	order_id, customer_id, order_date, status
payments	payment_id, order_id, method, amount, status, paid_at
products	product_id, product_name, category, price, stock_qty
stg_orders	order_id, customer_id, order_date, status

The Big Picture: Our dbt Architecture

We will follow this workflow:



| Customer |

| Sales |

| Marketing |

| Products |

| Finance |

+-----+

|

v

POWER BI

|

v

BUSINESS INSIGHT

What Is the Raw Layer?

The raw layer is the **original source data**. In our case, these tables already exist in Neon. We should normally avoid directly changing raw source tables. Raw data acts as our **original evidence** and gives us something to return to if a transformation goes wrong.

What Is the Staging Layer?

Staging is the first dbt transformation layer. A staging model is usually a clean, close-to-1-to-1 version of a raw source table. Typical staging work includes selecting relevant columns, renaming unclear columns, standardizing names and basic data preparation. We do not normally perform large business calculations here.

What Goes Inside a Staging File?

Example file:

```
models/  
└── staging/  
    └── stg_customers.sql
```

Inside the file:

```
SELECT  
    customer_id,  
    full_name,  
    email,  
    city,  
    state,  
    signup_date  
FROM {{ source('raw', 'customers') }}
```

Why Use source()?

The source() function tells dbt that the data comes from a raw/source table. For example:

```
{{ source('raw', 'customers') }}
```

This allows dbt to understand the source table and helps us keep the project organized.

Our Staging Folder

models/

```
└── staging/
    ├── stg_customers.sql
    ├── stg_orders.sql
    ├── stg_order_items.sql
    ├── stg_products.sql
    ├── stg_payments.sql
    ├── stg_exchange_rate.sql
    ├── stg_marketing_spend.sql
    └── schema.yml
```

Staging Model Examples

stg_orders.sql

```
SELECT
    order_id,
    customer_id,
    order_date,
    status
FROM {{ source('raw', 'orders') }}
```

stg_order_items.sql

```
SELECT
    order_item_id,
    order_id,
    product_id,
    quantity,
```

```
    unit_price  
FROM {{ source('raw', 'order_items') }}
```

stg_products.sql

```
SELECT  
    product_id,  
    product_name,  
    category,  
    price,  
    stock_qty  
FROM {{ source('raw', 'products') }}
```

stg_payments.sql

```
SELECT  
    payment_id,  
    order_id,  
    method,  
    amount AS payment_amount_ngn,  
    status AS payment_status,  
    paid_at  
FROM {{ source('raw', 'payments') }}
```

stg_marketing_spend.sql

```
SELECT  
    campaign_id,
```

```
campaign_name,  
channel,  
spend_date,  
clicks,  
conversions,  
amount_ngn AS marketing_spend_ngn  
FROM {{ source('raw', 'naijashop_marketing_ad_spend_csv') }}
```

stg_exchange_rate.sql

```
SELECT  
    "from" AS source_currency,  
    "to" AS target_currency,  
    rate,  
    amount,  
    result,  
    success,  
    change24h AS change_24h,  
    "updatedAt" AS updated_at,  
    changePct24h AS change_pct_24h  
FROM {{ source('raw', 'exchange_rate') }}
```

What Is the Intermediate Layer?

Intermediate models are where we begin combining staging models and performing reusable calculations. For example, we can join `order_items` to `products` and calculate the value of each line item. Then we can aggregate line items into order totals.

ref() — One of the Most Important dbt Concepts

The ref() function tells dbt that one dbt model depends on another dbt model.

```
{{ ref('stg_order_items') }}
```

This means: use the model named stg_order_items. dbt can then understand the dependency and build models in the correct order.

Function	Meaning	Example
source()	Gets data from an original/raw source	{{ source('raw', 'orders') }}
ref()	Gets data from another dbt model	{{ ref('stg_orders') }}

First Intermediate Model: Enrich Order Items

Create:

```
models/  
└── intermediate/  
    └── int_order_items_enriched.sql
```

Put this inside:

```
SELECT  
  
    oi.order_item_id,  
  
    oi.order_id,  
  
    oi.product_id,  
  
    p.product_name,
```

```

    p.category,
    oi.quantity,
    oi.unit_price,
    oi.quantity * oi.unit_price AS item_total_ngn
FROM {{ ref('stg_order_items') }} oi
LEFT JOIN {{ ref('stg_products') }} p
    ON oi.product_id = p.product_id

```

What happened? We combined order_items and products, then calculated quantity × unit_price. This is an intermediate transformation.

Second Intermediate Model: Order Totals

Create:

```
models/intermediate/int_order_totals.sql
```

```

SELECT
    order_id,
    SUM(item_total_ngn) AS order_total_ngn,
    SUM(quantity) AS total_items,
    COUNT(DISTINCT product_id) AS distinct_products
FROM {{ ref('int_order_items_enriched') }}
GROUP BY order_id

```

What Happened to the Data?

order_items

|

v

stg_order_items

|

v

int_order_items_enriched

|

v

int_order_totals

This is the dbt transformation pipeline: raw data is staged, intermediate calculations are performed, and the results are then used to create business-ready marts.

What Are Marts?

Marts are business-facing analytical models. Instead of being designed simply around source tables, they are designed around business questions and business areas.

- Customer mart — Who are our customers and how valuable are they?
- Sales mart — How many orders and how much order value do we have?
- Marketing mart — How much are campaigns spending and converting?
- Product mart — Which products and categories are selling?
- Finance/payment mart — Do payment records reconcile with order values?

First Mart: fct_orders

Create:

models/marts/fct_orders.sql

```
SELECT
  o.order_id,
  o.customer_id,
  o.order_date,
  o.status,
  COALESCE(t.order_total_ngn, 0) AS order_total_ngn,
  COALESCE(t.total_items, 0) AS total_items,
  COALESCE(t.distinct_products, 0) AS distinct_products
FROM {{ ref('stg_orders') }} o
LEFT JOIN {{ ref('int_order_totals') }} t
  ON o.order_id = t.order_id
```

What Is a Fact Table?

A fact table normally represents measurable business events. In NaijaShop, fct_orders represents orders and contains measures such as order_total_ngn and total_items.

What Is a Dimension?

A dimension describes an entity. Examples include dim_customers, dim_products and dim_date.

Customer Dimension

```
SELECT
    customer_id,
    full_name,
    email,
    city,
    state,
    signup_date
FROM {{ ref('stg_customers') }}
```

Customer 360

Create: models/marts/mart_customer_360.sql

```
WITH customer_metrics AS (
    SELECT
        customer_id,
        COUNT(*) AS total_orders,
        SUM(order_total_ngn) AS lifetime_value_ngn,
        AVG(order_total_ngn) AS average_order_value_ngn,
        MIN(order_date) AS first_order_date,
        MAX(order_date) AS last_order_date
```

```

FROM {{ ref('fct_orders') }}

GROUP BY customer_id

)

SELECT

    c.customer_id,

    c.full_name,

    c.email,

    c.city,

    c.state,

    c.signup_date,

    COALESCE(m.total_orders, 0) AS total_orders,

    COALESCE(m.lifetime_value_ngn, 0) AS lifetime_value_ngn,

    COALESCE(m.average_order_value_ngn, 0) AS average_order_value_ngn,

    m.first_order_date,

    m.last_order_date

FROM {{ ref('stg_customers') }} c

LEFT JOIN customer_metrics m

    ON c.customer_id = m.customer_id

```

Product Performance

```

WITH product_sales AS (

    SELECT

        product_id,

        SUM(quantity) AS units_sold,

        SUM(item_total_ngn) AS sales_value_ngn,

        COUNT(DISTINCT order_id) AS number_of_orders

```

```

FROM {{ ref('int_order_items_enriched') }}

GROUP BY product_id

)

SELECT

    p.product_id,

    p.product_name,

    p.category,

    p.price,

    p.stock_qty,

    COALESCE(s.units_sold, 0) AS units_sold,

    COALESCE(s.sales_value_ngn, 0) AS sales_value_ngn,

    COALESCE(s.number_of_orders, 0) AS number_of_orders

FROM {{ ref('stg_products') }} p

LEFT JOIN product_sales s

    ON p.product_id = s.product_id

```

Daily Revenue

```

SELECT

    order_date,

    COUNT(*) AS total_orders,

    COUNT(DISTINCT customer_id) AS unique_customers,

    SUM(order_total_ngn) AS order_value_ngn,

    AVG(order_total_ngn) AS average_order_value_ngn,

    SUM(total_items) AS total_items

FROM {{ ref('fct_orders') }}

```

GROUP BY order_date

Payment Reconciliation

Because NaijaShop has both orders and payments, we can compare the calculated order value with successful payment records. This is a useful real-world data engineering exercise.

```
SELECT
    order_id,
    SUM(
        CASE
            WHEN LOWER(payment_status) IN ('paid', 'successful', 'completed')
            THEN payment_amount_ngn
            ELSE 0
        END
    ) AS successful_payment_ngn,
    COUNT(*) AS payment_records
FROM {{ ref('stg_payments') }}
GROUP BY order_id
```

A reconciliation model can then classify records as Fully Paid, Underpaid, Overpaid, or No Payment Record. Verify the actual payment status values in your database before using those categories.

RFM Customer Segmentation

RFM means Recency, Frequency and Monetary Value.

Metric	Question
Recency	How recently did the customer order?
Frequency	How many orders has the customer made?
Monetary	How much order value has the customer generated?

```
WITH customer_metrics AS (
```

```
  SELECT
```

```
    customer_id,
```

```
    MAX(order_date) AS last_order_date,
```

```
    COUNT(*) AS frequency,
```

```
    SUM(order_total_ngn) AS monetary_value_ngn
```

```
  FROM {{ ref('fct_orders') }}
```

```
  GROUP BY customer_id
```

```
)
```

```
SELECT
```

```
  customer_id,
```

```
  last_order_date,
```

```
  CURRENT_DATE - last_order_date AS recency_days,
```

```
  frequency,
```

```
  monetary_value_ngn,
```

```
  CASE
```

```
    WHEN CURRENT_DATE - last_order_date <= 30
```

```
      AND frequency >= 5
```

```

        THEN 'Frequent Recent Customer'

    WHEN CURRENT_DATE - last_order_date <= 90

        AND frequency >= 2

        THEN 'Repeat Customer'

    WHEN CURRENT_DATE - last_order_date > 90

        THEN 'Needs Attention'

    ELSE 'Other'

END AS customer_segment

FROM customer_metrics

```

Marketing Performance

```

SELECT

    campaign_id,

    campaign_name,

    channel,

    SUM(clicks) AS total_clicks,

    SUM(conversions) AS total_conversions,

    SUM(marketing_spend_ngn) AS total_spend_ngn,

    ROUND(

        SUM(marketing_spend_ngn)

        / NULLIF(SUM(clicks), 0),

        2

    ) AS cost_per_click_ngn,

    ROUND(

        SUM(marketing_spend_ngn)

        / NULLIF(SUM(conversions), 0),

```

```

2
) AS cost_per_conversion_ngn,
ROUND(
    SUM(conversions)::numeric
    / NULLIF(SUM(clicks), 0)
    * 100,
    2
) AS conversion_rate_pct
FROM {{ ref('stg_marketing_spend') }}
GROUP BY
    campaign_id,
    campaign_name,
    channel

```

Important Marketing Attribution Lesson

Daily marketing spend can be compared with daily orders, but that does not automatically prove which campaign generated which order. To calculate true campaign-attributed revenue or customer acquisition cost, we need an attribution key such as `campaign_id` stored with a customer, order, or conversion event.

Month-over-Month Revenue Growth

```

WITH monthly_revenue AS (
    SELECT
        DATE_TRUNC('month', order_date)::date AS month,
        COUNT(*) AS total_orders,

```

```

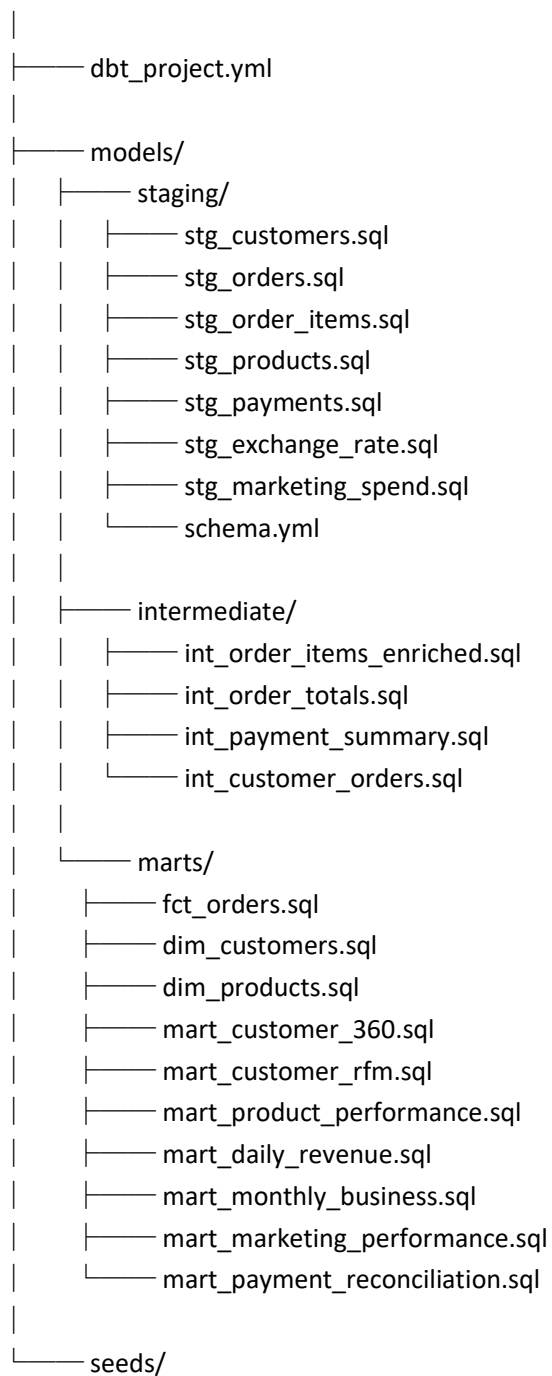
        SUM(order_total_ngn) AS revenue_ngn
    FROM {{ ref('fct_orders') }}
    GROUP BY 1
),
with_previous_month AS (
    SELECT
        month,
        total_orders,
        revenue_ngn,
        LAG(revenue_ngn) OVER (
            ORDER BY month
        ) AS previous_month_revenue_ngn
    FROM monthly_revenue
)
SELECT
    month,
    total_orders,
    revenue_ngn,
    previous_month_revenue_ngn,
    ROUND(
        (
            revenue_ngn - previous_month_revenue_ngn
        )
        / NULLIF(previous_month_revenue_ngn, 0)
        * 100,
        2
    ) AS revenue_growth_pct

```

FROM with_previous_month

The Complete Project Structure

naijashop/



The Five-Step dbt Workflow

SOURCE — Where is the raw data? In our project, the source data is in Neon PostgreSQL.

STAGE — Clean, rename and standardize the raw source tables.

INTERMEDIATE — Join staging models and perform reusable calculations.

MART — Create business-ready datasets that answer business questions.

TEST — Check whether the transformed data can be trusted.

Running dbt from Your Computer

Open PowerShell in the NaijaShop dbt project folder.

```
cd "C:\path\to\naijashop_dbt"
```

Run the models:

```
dbt run
```

Run the tests:

```
dbt test
```

dbt reads source() and ref() dependencies and determines the order in which models need to be built.

Example Dependency Flow

stg_products

|

+-----+

|

|

v

v

stg_order_items product data

|

v

int_order_items_enriched

|

v

int_order_totals

|

v

fct_orders

|

+-----+

|

|

v

v

mart_daily_revenue mart_customer_360

Data Quality Tests

Students should learn that transforming data is not enough. We also need to test whether the data is trustworthy.

version: 2

models:

- name: stg_customers

columns:

- name: customer_id

tests:

- unique
- not_null

- name: stg_orders

columns:

- name: order_id

tests:

- unique
- not_null

- name: customer_id

tests:

- not_null

Then run:

dbt test

Practical Student Exercise

1. Create models/staging/stg_customers.sql.

2. Create models/staging/stg_products.sql.
3. Create models/staging/stg_order_items.sql.
4. Create int_order_items_enriched.sql and calculate quantity × unit_price.
5. Create int_order_totals.sql and calculate order totals, total items and distinct products.
6. Create fct_orders.sql by combining orders with order totals.
7. Create mart_daily_revenue.sql to answer: How much did NaijaShop sell each day?
8. Create mart_customer_360.sql to answer: Who are our customers and how much have they purchased?
9. Create mart_marketing_performance.sql to analyze spend, clicks and conversions.
10. Run dbt run, then dbt test, and inspect the resulting models in Neon.

Important Questions Students Should Be Able to

Answer

- What is dbt?
- Why do we use dbt if we already have SQL?
- What is a source?
- What is a staging model?
- Why do we use staging models?
- What is an intermediate model?
- What is a mart?
- What is a fact table?

- What is a dimension table?
- What is the difference between source() and ref()?
- Why do we calculate order totals from order_items?
- Why do we test our dbt models?
- How does dbt know which model to build first?

37. Final Concept: What dbt Is Giving Us

The most important lesson is that dbt is not magic and it is not a replacement for SQL. We are still using SQL to transform data. dbt gives our SQL structure, dependencies, reusable models, tests, documentation and a repeatable build process.

SQL

+

Organized SQL files

+

Dependencies

+

Testing

+

Documentation

+

Reusable models

+

Lineage

+

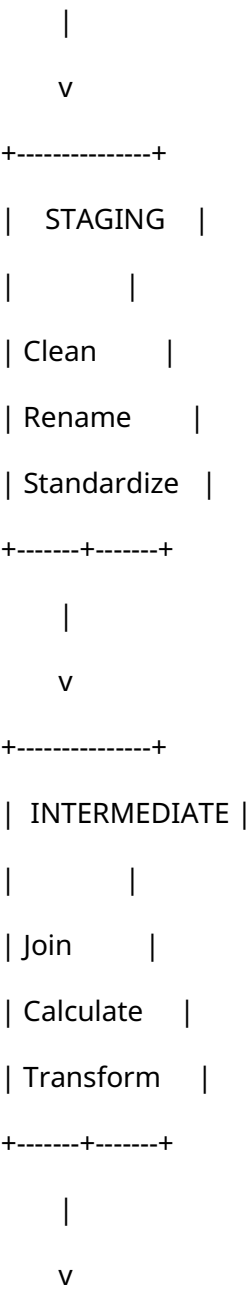
Repeatable builds

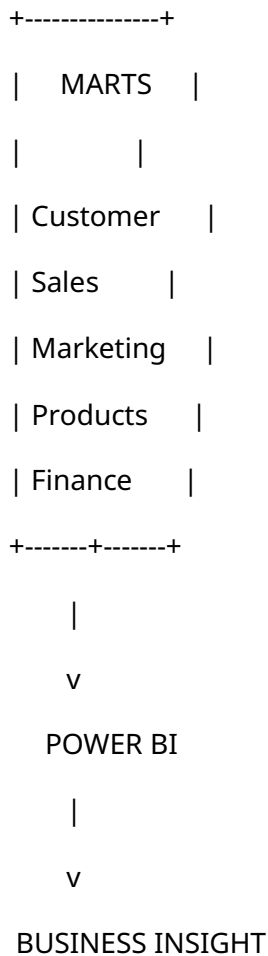
=

dbt Data Engineering Project

38. Final NaijaShop Workflow

NAIJASHOP RAW DATA





In Conclusion

We are not simply writing SQL queries. We are building a data transformation system. We start with raw data, clean it in staging, combine and calculate in intermediate models, turn the results into business-focused marts, test the data, and finally make the models available for reporting and analytics. This is the mindset of a data engineer.